Artificial Intelligence II: Deep learning methods

Lab Setup Guide

Vlad Vasilescu Ana Neacșu

Contents

1 Environment Setup **3**
- 1.1 Python & Virtual Environments . . . 3
- 1.2 Installing Packages . . . 4
- 1.3 Verifying Your Setup . . . 5

2 Jupyter Notebooks **5**
- 2.1 Working Offline . . . 5
- 2.2 Working Online – Google Colab . . . 6

3 Git Basics **6**
- 3.1 Git CLI commands . . . 7
- 3.2 Connect to GitHub . . . 7

1 Environment Setup

This guide is intended for setting up the environment to run the algorithms presented at Lab. We'll be using `Python 3.11` and `Jupyter notebooks`. If you prefer working without installing anything locally, you can jump directly to the Google Colab section.

Whichever route you choose, the workflow is the same:

1. Get Python 3.11 (directly, or through `conda`).
2. Create and activate an isolated virtual environment, called `lab_env` in this guide.
3. Install the required packages from `requirements.txt`.
4. Launch Jupyter and select the right kernel.

1.1 Python & Virtual Environments

Virtual environments in Python are isolated environments that enable you to manage and install dependencies for your projects separately from the system-wide Python installation. Below you can find two options. If you choose the `conda` option, you do **not** need to install Python separately, since `conda` provides it.

Option (recommended) #1: `conda`

Install Miniconda (or Anaconda), then create a new environment:

```
> conda create --name lab_env python=3.11
```

Activate it:

```
> conda activate lab_env
```

Deactivate the current environment:

```
> conda deactivate
```

Option #2: `venv`

You can find the installers for `Python 3.11` on the Python downloads page (look for the latest 3.11.x release). On Windows, make sure to tick *"Add Python to PATH"* in the installer. After installing, open a terminal and run

```
> python --version
```

to verify that the correct version is installed (on Linux and macOS you may need to use `python3` instead of `python`). If no Python executable can be found, see How to Add Python to PATH.

Create a new environment:

```
> python -m venv lab_env
```

Activate it:

- For Windows:

```
> lab_env\Scripts\activate
```

- For Linux and macOS:

```
> source lab_env/bin/activate
```

Deactivate the current environment:

```
> deactivate
```

1.2 Installing Packages

Once inside the virtual environment, you'll need to install the packages that your project needs. Packages in Python can be installed using the `pip` package manager:

```
> pip install <package_name>
```

or

```
> python -m pip install <package_name>
```

If you want to install a specific version of a package, you should do:

```
> pip install <package_name>==<package_version>
```

You can generate a list of dependencies for the current virtual environment by running:

```
> pip freeze > requirements.txt
```

This will create a file `requirements.txt` with all installed Python packages and their corresponding versions.

If you want to work on an existing project which provides an already generated `requirements.txt`, you can run the following:

```
> pip install -r requirements.txt
```

This will install all packages specified in the given file.

For the Lab: a `requirements.txt` with all the necessary packages for running the Lab notebooks is provided on the course's GitHub page. Download it, activate your `lab_env` environment, navigate (`cd`) to the folder containing the file, and run the command above.

1.3 Verifying Your Setup

To check that everything works, run the following inside the activated environment:

```
> python -c "import torch, numpy; print(torch.__version__, numpy.__version__,
    torch.cuda.is_available())"
```

If this prints without errors, you are ready to go. `torch.cuda.is_available()` will print `False` on machines without a supported GPU, which is fine for most Lab exercises. If something fails, copy the full output when asking for help.

2 Jupyter Notebooks

Jupyter notebooks are interactive computing environments that allow users to create and share documents containing live code, equations, visualizations, and narrative text. The name "Jupyter" is derived from the core programming languages it supports: Julia, Python, and R.

A notebook is comprised of *cells* which may include Python, Markdown or HTML code. A Python cell can be executed by `Ctrl+Enter` or `Shift+Enter`, or through the `Run` button from the interface. The output (if any) is immediately displayed under each cell.

If a notebook needs an extra or updated package, you can install it directly from a cell, for example:

```
!pip install <package_name>
```

and restart the kernel afterwards if the package was already imported.

2.1 Working Offline

In order to locally launch a Jupyter notebook, you'll need to have `Jupyter Lab` installed in your environment:

```
> pip install jupyterlab
```

Now you can open `Jupyter Lab` by running:

```
> jupyter lab
```

in your working folder. This will automatically open it in your browser, and in the console it will output something like:

```
To access the server, open this file in a browser:
    file:<some_path>.html
Or copy and paste one of these URLs:
    http://localhost:8888/lab?token=<some_chars>
    http://127.0.0.1:8888/lab?token=<some_chars>
```

The URLs can be copy-pasted into a browser of your choice to open the `Jupyter Lab` interface.

Kernel note: always launch Jupyter from inside the activated `lab_env` environment, otherwise the notebook will not see the packages you installed. If you still get `ModuleNotFoundError`, register the environment as a kernel:

```
> python -m ipykernel install --user --name lab_env
```

and select the `lab_env` kernel from the `Kernel > Change Kernel` menu of the notebook.

You can check out this tutorial on how to get started with Jupyter.

2.2 Working Online – Google Colab

Google Colab is a free cloud service provided by Google that allows users to write and execute Python code through the browser. One of its most important features is the free access to GPU and TPU (Tensor Processing Unit) resources, which we'll make use of during Lab. You can check out their tutorial.

To enable a GPU, go to `Runtime > Change runtime type` and select a GPU as the hardware accelerator.

To successfully execute the notebooks we'll be using at Lab, be sure to include all the other `.py` files, containing implemented dependencies, and the other folders with images or data. You could set up a folder in your Google Drive and mount it in the notebook:

```
from google.colab import drive
drive.mount('/content/drive')
```

Or, after uploading your notebook (`.ipynb` file) here, you can manually add all the other files by accessing the `Files > Upload` section in the left menu.

Note: Simply uploaded external files will be deleted after some time, with the runtime being considered idle. In this case, it is recommended to progressively save your work by mounting a folder from your Google Drive.

3 Git Basics

Git is a distributed version control system used for tracking changes in source code during software development. When you're building an open source project, you'll need a way to document or track your code and to keep your work organized, which is what Git does.

GitHub, on the other hand, is a web-based platform for hosting Git repositories and collaborating with others on software projects. GitHub uses Git underneath, letting you manage your local Git repositories.

You can install Git on your machine by following these steps. Before your first commit, tell Git who you are:

```
> git config --global user.name "Your Name"
> git config --global user.email "you@example.com"
```

3.1 Git CLI commands

In order to use Git on one of your projects, navigate to the project's folder, open a terminal and run:

```
> git init
```

which initializes an empty Git repository in the project's directory. To add files to the staging area (prepare them for committing), use the following command:

```
> git add <file_name>
```

which will include the files in your Git repo for tracking. To see which files are included in your Git repo you can run:

```
> git status
```

This will list all tracked and untracked files, files staged for *commit* and files from your repo which have been modified (i.e. the current status of your repo). When you want to take a snapshot of your project, you can do:

```
> git commit -m "Changed file <x>"
```

which creates a checkpoint in the *commit tree*, encapsulating all changes that have been made relative to the previous commit. To see all previous commits, run:

```
> git log
```

which prints out the previous commits' details (hash, message, date, author).

Not everything belongs in a repository. To keep files such as virtual environments (`lab_env/`) or large datasets out of Git, list them in a file named `.gitignore` in the root of your project, one pattern per line.

In a scenario when you want to add a new feature or fix some bugs, the usage of *branches* comes in very handy. To learn how Git branching works, we recommend checking the interactive tutorial *Learn Git Branching*.

3.2 Connect to GitHub

Create a GitHub account here and create a new repo, choosing a name, a description and its visibility settings.

In the folder where your local repo is initialized, you can add the newly created GitHub repo as a *remote repository*:

```
> git remote add <alias_name> <repository_URL>
```

Usually, the main remote repository has the alias `origin`. You can add multiple remote repositories by using different aliases.

Before transferring your work to the remote, you need to first check if there were any changes made on the remote in the meantime. To do so, run:

```
> git fetch <alias_name>
```

To visualize the *difference* between your current local branch and the remote branch (i.e. if there were any changes made on the remote but not on the local), you would do:

```
> git diff <alias_name>/<remote_branch_name>
```

which will print out all differences between your local branch (where `diff` was run) and the remote branch of choice (after doing `fetch`).

To combine your local branch with the changes made on the remote, run:

```
> git merge <alias_name>/<remote_branch_name>
```

Another choice for combining two branches is through *rebase*.

The `fetch` and `merge` steps can be executed at once by running:

```
> git pull <alias_name> <remote_branch_name>
```

which will prompt you to resolve any existing conflicts.

When all conflicts have been resolved and all desired changes made, you would want to push your local changes up to the remote repo:

```
> git push <alias_name> <remote_branch_name>
```

which will push all local changes to the remote repo at the given branch. The first time you push a new branch, use `git push -u <alias_name> <remote_branch_name>` so that later you can simply run `git push`. The default branch is usually called `main` (older repositories use `master`).

Authentication: GitHub no longer accepts account passwords for `git push` over HTTPS. You will need either a personal access token (used in place of the password) or an SSH key.

If you just want to work on an existing project hosted on GitHub you could do:

```
> git clone <repository_URL>
```

and all corresponding remote resources will be copied in a new local folder, where you can start working.

For an in-depth discussion regarding advanced Git tactics you can check Atlassian's Git tutorials and Toptal's Advanced Git Guide.